

UNIVERSIDAD NACIONAL DE ENTRE RÍOS

Facultad de Ingeniería

Carrera: T.U.P.E.D

Materia: Fundamentos de Algoritmos y Estructuras de Datos

Título del Trabajo: Aplicaciones de TADs

Equipo del Docente:

Dr. Javier E. Diaz Zamboni

Bioing. Jordán F. Insfrán

Bioing. Diana Vertiz del Valle (de licencia)

Esp. Bioing. Juan Francisco Rizzato

Dr. Bioing. Feliciano Franco

Integrantes del equipo:

Franco Acevedo

Franco Tomiozzo

Comisión: 1

Fecha de entrega: 04/05/2026

Link del repositorio:

<https://github.com/RENCO12BOT/AyED2026c1-Acevedo-Tomiozzo-.git>

Problema N° 3 Algoritmos de Ordenamiento en Python

INTRODUCCIÓN

El análisis de los tres algoritmos de ordenamiento implementados en Python: Burbuja, Quicksort y Radix Sort. Para cada algoritmo se describe su lógica de funcionamiento, se presentan los métodos clave con sus bloques de código y se justifica la complejidad temporal en cada escenario relevante, si están en el orden incorrecto. Este proceso se repite reduciendo el rango de comparación en cada pasada, ya que el elemento más grande siempre "burbujea" hacia el final.

DESARROLLO

Burbuja

El algoritmo burbuja compara elementos adyacentes e intercambian sus posiciones si están desordenados. Este proceso se repite en varias pasadas hasta que la lista queda ordenada, haciendo que los valores más grandes se desplacen al final.

La implementación utiliza un bucle externo para las pasadas y uno interno para las comparaciones. Aunque se reduce el rango en cada iteración, no se detecta si la lista ya está ordenada.

Método ordenar_lista() ($O(n^2)$)

El método recorre la lista en dos niveles: el bucle externo (while) controla cuántas pasadas se realizan, y el bucle interno (for) ejecuta las comparaciones e intercambios en cada pasada

```

class Burbuja:
    def __init__(self, lista):
        self.__lista = lista

    def ordenar_lista(self):
        """Ordena la lista de manera ascendente usando burbuja."""
        contador = 0
        while contador < len(self.__lista):
            tamaño_lista = len(self.__lista) - contador
            for i in range(1, tamaño_lista):
                if self.__lista[i] < self.__lista[i - 1]:
                    aux = self.__lista[i]
                    self.__lista[i] = self.__lista[i - 1]
                    self.__lista[i - 1] = aux
            contador += 1
        return self.__lista

    def __str__(self):
        return f"{self.__lista}"

```

Justificación: El algoritmo realiza múltiples comparaciones en cada pasada, acumulando aproximadamente $n(n-1)/2$ operaciones. Por esta razón, su complejidad es $O(n^2)$ en todos los casos. No incluye una optimización para detectar listas ya ordenadas, por lo que el mejor caso no mejora. El espacio es $O(1)$ porque solo utiliza variables auxiliares.

Quicksort

Quicksort es un algoritmo de división y conquista. Elige un pivote, coloca todos los elementos menores a su izquierda y los mayores a su derecha (partición), y luego aplica el mismo proceso recursivamente sobre cada sublista. Esta implementación usa un pivote aleatorio para evitar el peor caso en listas ya ordenadas.

Método ordenar() ($O(n \log n)$ promedio)

El método público es la entrada recursiva del algoritmo. Gestiona los índices de inicio y fin del segmento actual y delega la partición al método privado `__ubicar_pivote()`.

```
def ordenar(self, indice_inicio=None, indice_final=None, lista=None):
    """
    Implementa el algoritmo de Quicksort para ordenar la lista.
    Si no se pasan índices ni lista, usa los valores por defecto del objeto.
    """
    if lista is None:
        lista = self.__lista

    if indice_inicio is None:
        indice_inicio = 0
    if indice_final is None:
        indice_final = len(lista) - 1

    if indice_inicio < indice_final:
        # Ubica el pivote en su posición correcta
        posicion_pivote = self.__ubicar_pivote(indice_inicio, indice_final, lista)

        # Ordena recursivamente las sublistas izquierda y derecha
        self.ordenar(indice_inicio, posicion_pivote - 1, lista)
        self.ordenar(posicion_pivote + 1, indice_final, lista)

    return self.__lista
```

Método __ubicar_pivote() (O(n))

Selecciona el pivote aleatoriamente dentro del rango actual y lo mueve al inicio del segmento. Luego recorre el segmento con dos índices convergentes, intercambiando elementos fuera de lugar, hasta posicionar el pivote definitivamente.

```
def __ubicar_pivote(self, indice_inicio, indice_final, lista):
    """
    Posiciona correctamente el pivote en la lista.
    MODIFICADO: el pivote se elige aleatoriamente para evitar el peor
    caso O(n²) en listas ya ordenadas y reducir el riesgo de RecursionError.
    Todos los elementos menores que el pivote quedan a la izquierda
    y los mayores a la derecha.
    """
    # MODIFICADO: pivote aleatorio en lugar de siempre el primero
    pivote_idx = random.randint(indice_inicio, indice_final)
    lista[indice_inicio], lista[pivote_idx] = lista[pivote_idx], lista[indice_inicio]

    pivote = lista[indice_inicio]
    izquierda = indice_inicio + 1
    derecha = indice_final

    while True:
        # Recorremos desde la izquierda hasta encontrar algo > pivote
        while izquierda <= derecha and lista[izquierda] <= pivote:
            izquierda += 1

        # Recorremos desde la derecha hasta encontrar algo < pivote
        while izquierda <= derecha and lista[derecha] >= pivote:
            derecha -= 1

        if izquierda <= derecha:
            # Intercambiamos los elementos fuera de lugar
            lista[izquierda], lista[derecha] = lista[derecha], lista[izquierda]
        else:
            break

    # Colocamos el pivote en su posición final
    lista[indice_inicio], lista[derecha] = lista[derecha], lista[indice_inicio]
    return derecha
```

Justificación: el pivote aleatorio distribuye estadísticamente el trabajo de manera equilibrada. En promedio, cada nivel de recursión procesa $O(n)$ elementos en $O(\log n)$ niveles, dando $O(n \log n)$. Sin pivote aleatorio, una lista ya ordenada provocaría particiones de tamaño 1 y $n-1$, degenerando a $O(n^2)$ y potencialmente causando RecursionError en Python.

Radix Sort

Radix Sort es un algoritmo no comparativo que ordena los números dígito a dígito, desde el menos significativo (LSD) hasta el más significativo. Utiliza baldes (buckets) del 0 al 9 para distribuir los elementos según el dígito de cada posición.

Método ordenar() ($O(n \cdot k)$)

El método calcula primero cuántos dígitos tiene el número más grande (k), normaliza todos los elementos a esa longitud con ceros a la izquierda, y luego ejecuta k pasadas de distribución y aplanamiento.

```
def ordenar(self):
    # Determina la cantidad máxima de dígitos en los números de la lista.
    cantidad_de_digitos = 0
    for elemento in self.__lista:
        digitos = len(str(elemento))
        if digitos > cantidad_de_digitos:
            cantidad_de_digitos = digitos

    # Normaliza la lista de números como cadenas con ceros a la izquierda.
    lista_normalizada = [
        str(elemento).zfill(cantidad_de_digitos) for elemento in self.__lista
    ]

    # Recorre cada posición de dígito desde el menos significativo al más significativo.
    for posicion in range(cantidad_de_digitos - 1, -1, -1):
        # Crea 10 "baldes", uno por cada dígito 0-9
        lista_auxiliar = [[] for _ in range(10)]

        # Distribuye según el dígito en la posición actual
        for elemento in lista_normalizada:
            digito = int(elemento[posicion])
            lista_auxiliar[digito].append(elemento)

        # Aplana la lista auxiliar en una sola lista
        lista_normalizada = [
            elemento for sublista in lista_auxiliar for elemento in sublista
        ]

    # Convierte de vuelta a enteros
    self.__lista = [int(elemento) for elemento in lista_normalizada]
    return self.__lista
```

Justificación: el algoritmo realiza exactamente k pasadas sobre la lista, y en cada pasada distribuye los n elementos en los 10 baldes y luego los aplanan. Cada pasada tiene costo $O(n)$.

El costo total es $O(n \cdot k)$, donde k es el número de dígitos del valor máximo. Como k es constante respecto al tamaño del dominio (por ejemplo, $k = 10$ para números de hasta diez dígitos), en la práctica el algoritmo se comporta como $O(n)$.

CONCLUSIONES

El análisis de los tres algoritmos permite extraer las siguientes conclusiones:

- Burbuja es el algoritmo más simple pero el menos eficiente. Su complejidad $O(n^2)$ en todos los casos lo hace inadecuado para listas grandes. Es útil con fines educativos o con listas muy pequeñas.
- Quicksort es el algoritmo de uso general más equilibrado: $O(n \log n)$ en promedio y espacio auxiliar $O(\log n)$. El pivote aleatorio implementado evita el peor caso y lo hace robusto frente a entradas adversas como listas ya ordenadas.
- Radix Sort ofrece $O(n \cdot k)$ determinista, independiente del orden inicial. Es la mejor opción cuando los elementos son enteros no negativos con un número de dígitos acotado, superando a Quicksort en esos contextos al evitar completamente las comparaciones.
- Desde el punto de vista del espacio, Burbuja es el más austero ($O(1)$), seguido de Quicksort ($O(\log n)$ por la pila de recursión) y Radix Sort ($O(n)$ por la lista normalizada y los baldes).

La elección del algoritmo óptimo depende del contexto: para conjuntos de enteros no negativos grandes, Radix Sort es preferible; para uso general, Quicksort con pivote aleatorio es la opción más versátil; Burbuja queda reservado para propósitos pedagógicos.